

C-CLARKE INTERNATIONAL INSTITUTE OF DIGITAL SCIENCES

Development Lessons Learned

24 real technical hiccups encountered while building Smart Hire, and how each was diagnosed and fixed.

Program	Diploma in Advanced AI & Software Engineering
Batch	25/26
Project	Smart Hire — Service Marketplace
Team	Nilan Iddawela, Janaka Eranda, Dasun Anjana, Damith Lasantha, Matheesha Milan
Date	21 July 2026

A running log of real technical problems the team hit while building Smart Hire, and exactly how each one was found and fixed. Kept in encounter order so later lessons build on earlier ones.

1. AI Output Must Not Directly Perform a Customer Booking

The Hiccup: Letting a chat response create a booking would allow an untrusted model response to make a business transaction without the customer selecting the date/time or confirming the request.

The Fix: The assistant only provides guidance and recommendations. Its Angular page displays a confirmation panel where the authenticated customer chooses a recommended service and a future date/time. It then calls the existing `POST /api/bookings` endpoint, keeping customer-role validation, active-service verification, provider ownership, and notification logic intact.

2. Conversation History Needs Ownership, Ordering, and a Parent Touch

The Hiccup: Saving messages alone isn't enough for usable chat history. Without an owner filter, users could inspect another user's messages; without updating the conversation timestamp, the history list is ordered by creation time rather than recent activity.

The Fix: Every conversation query filters by the authenticated `user_id`; a mismatched conversation returns 404. The chat route stores both messages in one transaction and updates the parent conversation's `updated_at`. Static routes such as `/conversations/analysis` are registered before `/conversations/{{conversation_id}}` so FastAPI doesn't try to parse “analysis” as an integer id.

3. The Endless Loading Spinner

The Hiccup: When simulating a payment or loading dashboard statistics, the UI would show a spinning wheel forever, even after the backend had sent the data back correctly.

The Fix: Imported `ChangeDetectorRef` and manually told Angular to redraw the screen with `detectChanges()` once the data arrived.

4. The Database Rejected Our Test Payments (Foreign Key Error)

The Hiccup: Submitting a payment on the dummy payment page crashed the backend with a 500 error.

The Fix: The payment page was hardcoded to pay for “Booking #1”, which no longer existed after the database was wiped. Built a dedicated data seeder that generates real users and bookings with known ids, and pointed the payment page at those ids.

5. The Backend Server Failed to Restart (Address Already in Use)

The Hiccup: The FastAPI backend would occasionally refuse to start, claiming port 8000 was already in use.

The Fix: A previous process hadn't been shut down cleanly. Made sure to explicitly kill old background server processes before starting a new one.

6. The Angular Application Crashed on Boot (NullInjectorError)

The Hiccup: Navigating to the Admin Dashboard for the first time crashed the entire app with a `NullInjectorError` for `HttpClient`.

The Fix: Modern standalone Angular no longer uses `HttpClientModule`. Added `provideHttpClient()` to the application's root providers in `app.config.ts`.

7. Duplicate Data Crashing the Seeder

The Hiccup: Clicking “Populate Presentation Data” twice threw a large error and stopped the seeder halfway through.

The Fix: The database enforces a unique `transaction_id`; inserting `TXN-1001` twice correctly triggered a `UniqueViolation`. Fixed the UX by locking the button to show “Database Populated” after the first successful click.

8. FastAPI Route Prioritization Conflicts

The Hiccup: Adding public list endpoints like `GET /api/reviews/` threw 422 errors, with FastAPI trying to parse the static path as a `{{review_id}}` integer.

The Fix: FastAPI matches routes in the order they're defined. Reordered so static routes (e.g. `GET /api/reviews/`) are declared before dynamic ones (`GET /api/reviews/{{review_id}}`).

9. Landing Page Styles Exceeded the Production Limit

The Hiccup: The first landing-page build exceeded Angular's 8 KB component-style budget by 148 bytes.

The Fix: Removed nonessential backdrop filters and decorative declarations, keeping the responsive layout intact. The production build passed without raising the project's budget.

10. The Endless Loading Spinner, Actually Explained

The Hiccup: The same spinner bug kept reappearing in new components after the first fix, because the real cause was never pinned down — it was blamed on `setTimeout()` rather than change detection itself.

The Fix: This project has no `zone.js` installed and never opts into zoneless mode either. Mutating a plain class field inside an `HttpClient.subscribe()` callback never triggers a re-render, since it's neither a signal write nor a DOM event. Every place doing this needs `ChangeDetectorRef.detectChanges()` in both the success and error branches. Confirmed by reading live computed styles in a browser session rather than guessing.

11. A `<td>` Cannot Safely Be `display:flex`

The Hiccup: A user reported a table's last column “looked a little off” — the row divider lines were jagged under some rows.

The Fix: A table cell's height normally matches the tallest cell in its row, but `display: flex` directly on a `<td>` breaks that, so an empty action cell collapsed to its own content height. Fixed by wrapping the buttons in an inner `<div>` and putting the flex styling there instead.

12. A Response Schema Missing a Field Is Invisible From the Frontend

The Hiccup: The admin payments table's date column stayed blank even though every payment genuinely had a `created_at` timestamp in the database.

The Fix: The `AdminPaymentResponse` Pydantic schema never listed `created_at` / `updated_at`, so FastAPI silently dropped them before the frontend ever saw them. When a real DB value “isn't showing up”, check the response schema before touching any frontend code.

13. One Stale Test File Can Fail the Entire Test Build

The Hiccup: `ng test` was failing before any of the team's own tests even ran.

The Fix: Angular's test builder compiles every spec file as one program. Six pre-existing spec files imported a component under its old, pre-rename name — a single bad import anywhere blocks the whole build. Fixed all six.

14. Provider Profile Verification Status Blank due to a 404

The Hiccup: The Provider Profile page showed a blank box for the verification status chip.

The Fix: A newly registered provider had no `ProviderProfile` row yet, so the profile endpoint threw a 404 and the Angular subscription failed silently. Fixed by auto-creating a default, unverified profile on first access.

15. Account Status Changes Not Syncing (401 Rejection)

The Hiccup: When an admin suspended or deactivated a user, that user's own Profile page still showed “ACTIVE”.

The Fix: `get_current_user` was rejecting non-active users with a 401 on `GET /api/auth/me`, so the frontend fell back to stale cached data. Removed the status check from `get_current_user` so status can always be read; real enforcement moved to a dedicated `require_active_user` dependency on action endpoints.

16. Suspended Users Could Still Act After the Status-Sync Fix

The Hiccup: After fixing Lesson 15, suspended and deactivated users could still create bookings, payments, and reviews, since `get_current_user` no longer blocked them.

The Fix: Added `require_active_user`, applied to every write/action endpoint, with a clear 403 message. `require_roles` now chains through it so role-protected endpoints are status-gated automatically.

17. TypeScript TS2353: password Missing From UserProfileUpdate

The Hiccup: Adding a password reset form caused a compile error: the property didn't exist on the update interface.

The Fix: Added `password?: string | null` to the frontend interface, plus the matching optional field and hashing logic on the backend schema and service.

18. Admin Changes Not Visible to Users

The Hiccup: Toggling a user's email verification or account status in the admin panel didn't reflect on that user's own Profile or Dashboard, even after a refresh.

The Fix: Two causes: the backend was still blocking non-active users from `GET /api/auth/me` (see Lesson 15), and the Customer Profile page was reading a cached signal instead of force-refreshing from the server on load.

19. The Zoneless Change-Detection Gap Keeps Coming Back

The Hiccup: The same root cause from Lessons 3 and 10 turned up twice more: toast messages never appeared anywhere in the app, and a wrong password on login just left the spinner running forever.

The Fix: Same fix every time — `ChangeDetectorRef.detectChanges()` after any plain-field mutation inside an async callback. Lesson: any new component built with plain fields and an async callback needs this by default, rather than waiting for a bug report.

20. A Backend Restart Can Wedge Mid-Shutdown During a Real DB Blip

The Hiccup: Frontend tests making real HTTP calls to `/admin/*` suddenly all hung for the full timeout, with no code changes to explain it.

The Fix: `uvicorn's --reload` watches the whole backend directory; adding a new folder triggered an unrelated restart that overlapped with a transient Supabase connectivity blip, wedging the server alive-but-unresponsive. Diagnosed by comparing a DB-touching health check against a non-DB one, and fixed by force-restarting the process.

21. A Feature Can Be "Built" and Still Not Do the One Thing That Matters

The Hiccup: The AI assistant's chat replies never mentioned real services, prices, or providers — just vague category names — even though a separate “Recommend a service” button on the same page worked fine.

The Fix: Two different code paths, only one of them database-backed. When something “isn't working right”, check whether the code path actually touches the database at all before assuming the logic itself is wrong.

22. A Bad SCSS Edit Can Silently Re-Parent Sibling Rules

The Hiccup: Adding one new CSS rule accidentally nested three unrelated, already-existing rules inside the new selector instead of their original parent, because a find-and-replace-style edit inserted a closing brace in the wrong place. Nothing errored.

The Fix: Always re-read a file after any SCSS edit that touches brace structure, and confirm sibling rules are still nested under the same parent as before — a compiler accepting the file isn't the same as it meaning what was intended.

23. Stock/Placeholder Images Need to Be Looked At, Not Just Fetched

The Hiccup: A first pass of category stock photos all resolved as valid URLs, but several were poor matches once actually viewed — a random tools-drawer photo for “Plumbing”, a generic laptop-typing photo for “Tech Support”.

The Fix: A URL returning 200 only proves the link isn't broken, not that the content is a good match. Every candidate image was downloaded and viewed, and re-picked if a human glancing at it wouldn't recognize the category. Preferred specific, individually-verified photo ids over a randomizing placeholder service.

24. Schema-Qualify Queries Against a Supabase Database

The Hiccup: A quick sanity-check query after adding a new `messages` table returned a confusing result — duplicate `id` columns and columns never defined anywhere in the codebase.

The Fix: Not corruption: Supabase's built-in Realtime feature has its own internal `realtime.messages` table in a different schema, and an unqualified `information_schema.columns` query matches across every schema. Re-running with `table_schema = 'public'` showed the new table was exactly as defined.